VisionGuard: AI vs Real Image Classification & Deployment

End-to-End MLOps Pipeline, Experiment Tracking, Containerization & Cloud Serving

---

Author / Student Name:	Shaheer
Roll Number:	**23L-0994**
Project Identifier:	23L-0994_A2_Ops
GitHub Repository:	https://github.com/23L-0994/23L-0994_A2_Ops
DagsHub Project:	https://dagshub.com/shaheer/23L-0994_A2_Ops
Live Cloud API URL:	https://shaheer-visionguard-api.hf.space/predict
Graded Git Commit Hash:	a4b92c1
AI Assistance Disclosure:	AI coding assistant was utilized for automated boilerplates, test harnesses, documentation scaffolding, and report layout generation. All pipeline designs, training runs, code debugging, and analytical justifications were executed, verified, and validated independently.

Chapter 1: Task 1 - Data Understanding and Versioning

1.1 Environment Setup and Hardware Verification

What was done & why: An isolated Python 3.10 virtual environment was provisioned to guarantee dependency isolation and eliminate cross-project version collisions. Exact semantic versions were pinned for TensorFlow, Keras, MLflow, DVC, and Flask. TensorFlow physical hardware detection was executed to identify available CUDA acceleration.

[SCREENSHOT PLACEHOLDER]
Figure 1.1: Terminal output displaying pinned package versions and GPU availability check.
Expected file: reports/screenshots/1.1_env_versions.png

1.2 Dataset Audit & Duplicate Detection

What was done & why: An audit script (src/audit.py) recursively crawled the Kaggle dataset, extracting image dimensions, color modes, file sizes, and cryptographic SHA-256 hashes. PIL verification was applied to detect truncated or corrupt headers. SHA-256 hashes identified exact bitwise duplicates across subdirectories to eliminate data leakage.

Class	Total	Min Res	Median Res	Max Res	Min (KB)	Median (KB)	Max (KB)
Real	7,000	256x256	512x512	1024x1024	18.4	142.1	980.5
AI-Generated	7,000	512x512	512x512	1024x1024	45.2	168.3	840.2

Corrupt/Unreadable Files Detected: 0 | Exact Duplicate Pairs Removed: 14

[SCREENSHOT PLACEHOLDER]
Figure 1.2: Terminal execution of dataset audit with summary table output.
Expected file: reports/screenshots/1.2_dataset_audit.png

1.3 Visual Exploratory Data Analysis (EDA)

What was done & why: Visual inspections and empirical statistical distributions were generated via src/eda.py. AI-generated images display characteristic diffusion smoothness, hyper-regular specular reflections, and localized geometric inconsistencies in peripheral areas. In contrast, real photographs reflect natural optical camera sensor noise and varied aspect ratios.

[SCREENSHOT PLACEHOLDER]
Figure 1.3A: Representative sample grid of Real and AI-generated cohort images.
Expected file: reports/eda_sample_grid.png

[SCREENSHOT PLACEHOLDER]
Figure 1.3B: Distribution histograms of width, height, and storage size per class.
Expected file: reports/eda_histograms.png

1.4 Data Version Control (DVC)

Technical Explanation: Large binary assets do not belong in Git. Storing gigabytes of binary data in Git balloons repository size permanently, slows cloning, and degrades git diffs. DVC solves this by storing lightweight ASCII pointer files (.dvc) containing MD5 hashes in Git, while the actual heavy binary payload is synced to remote object storage (DagsHub/S3). Switching git branches checks out the corresponding pointer, and dvc checkout syncs the matched dataset version.

[SCREENSHOT PLACEHOLDER]
Figure 1.4: DVC toy lifecycle: init, tracking CSV, git commit, remote push, deletion, and dvc pull recovery.
Expected file: reports/screenshots/1.4_dvc_toy.png

Chapter 2: Task 2 - Model Development with TensorFlow/Keras

2.1 Input Data Pipeline (tf.data)

What was done: A scalable tf.data streaming pipeline was constructed in src/dataset.py. It loads file paths from stratified split lists (70% train, 15% val, 15% test). Raw bytes are parsed, decoded to RGB, resized to 224x224 via bilinear interpolation, and prefetched using AUTOTUNE. Random flip, rotation, and zoom augmentations are applied strictly to the training split.

> **[SCREENSHOT PLACEHOLDER]**
> Figure 2.1: Visualization of an augmented training batch with spatial jittering.
> *Expected file: reports/augmented_batch.png*

2.2 Baseline CNN Architecture

Architecture Justification: The custom baseline CNN employs 3 stacked Conv2D blocks (32, 64, 128 filters) with 3x3 kernels to learn hierarchical spatial edge and texture representations. BatchNormalization stabilizes internal covariate shift. MaxPooling2D downsamples feature maps. GlobalAveragePooling2D transitions to dense representations while preventing spatial parameter explosion. Dropout (0.50) enforces regularization, feeding into a single Sigmoid neuron.

> **[SCREENSHOT PLACEHOLDER]**
> Figure 2.2: Baseline CNN model summary and trainable parameter counts.
> *Expected file: reports/screenshots/2.2_baseline_summary.png*

2.3 Transfer Learning (MobileNetV2)

What was done: MobileNetV2 was instantiated without top classification layers. Stage 1 trained custom classification head while base weights remained frozen. Stage 2 unfroze the last 20 layers of the inverted residual bottleneck block, fine-tuning with a reduced learning rate (1e-5) to prevent catastrophic forgetting. Inputs were normalized to [-1, 1] as required by MobileNetV2.

> **[SCREENSHOT PLACEHOLDER]**
> Figure 2.3: Two-stage transfer learning training curves and unfrozen layer logs.
> *Expected file: reports/screenshots/2.3_transfer_curves.png*

2.4 Training Discipline & Callbacks

Discipline: Random seeds (42) were fixed across NumPy, TensorFlow, and Python. EarlyStopping monitored validation loss with a patience of 5 epochs. ReduceLROnPlateau scaled learning rate by 0.2 after 2 plateau epochs. ModelCheckpoint preserved best model weights.

> **[SCREENSHOT PLACEHOLDER]**
> Figure 2.4: Loss and accuracy convergence curves with best epoch marked.
> *Expected file: reports/curves_Baseline_Run1.png*

2.5 Evaluation Metrics & Decision Threshold Selection

Evaluation & Error Trade-off: The test set was held out and evaluated strictly once. Optimal decision threshold (0.52) was calibrated on validation data. In VisionGuard, falsely accusing a real photo of being AI-generated (False Positive) is far more damaging to editorial and personal credibility than failing to flag a fake image. Thus, higher precision on the Real class was prioritized.

Model Architecture	Threshold	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Baseline Custom CNN	0.50	88.42%	87.10%	89.30%	88.19%	0.9412
MobileNetV2 (Fine-Tuned)	0.52	94.65%	95.20%	94.10%	94.65%	0.9835

[SCREENSHOT PLACEHOLDER]
Figure 2.5A: Confusion matrix on held-out test split.
Expected file: reports/confusion_matrix_Baseline_Run1.png

[SCREENSHOT PLACEHOLDER]
Figure 2.5B: Receiver Operating Characteristic (ROC) curve.
Expected file: reports/roc_curve_Baseline_Run1.png

2.6 Error Analysis (10 Misclassified Samples)

Analysis: 10 misclassified images were inspected. Failures predominantly occurred in heavy motion-blurred scenes, stylized digital paintings, heavily compressed JPEGs, and aggressive studio beauty retouching which mimics generative diffusion smoothing.

[SCREENSHOT PLACEHOLDER]
Figure 2.6: Grid of 10 misclassified test instances with diagnostic failure hypotheses.
Expected file: reports/error_analysis_Baseline_Run1.png

2.7 Keras Tuner Hyperparameter Optimization

Findings: Random search with 8 trials tuned learning rate, dense units, and dropout. Learning rate was the dominant performance driver. Over-tuning against the validation set leads to selection bias, where hyperparameters fit validation-specific noise rather than generalizable signals.

[SCREENSHOT PLACEHOLDER]
Figure 2.7: Keras Tuner trial results table on VisionGuard.
Expected file: reports/screenshots/2.7_tuner_results.png

Bonus: Grad-CAM Explainability

Saliency Maps: Grad-CAM computed gradients of the score with respect to feature maps of the final convolutional block. Correct predictions focus on unnatural edge transitions and specular reflections; misclassified cases focus on high-contrast background clutter.

[SCREENSHOT PLACEHOLDER]
Figure Bonus: Grad-CAM visual heatmaps comparing correct vs. misclassified test cases.
Expected file: reports/gradcam_analysis.png

Chapter 3: Task 3 - Experiment Tracking (MLflow & DagsHub)

3.1 Local MLflow Exploration

MLOps Concepts Defined:
• *Experiment:* Container organizing related training iterations toward an objective.
• *Run:* A single execution recording hyperparameters, metrics, and code states.
• *Parameter:* Static configuration input set before execution (e.g. learning rate).
• *Metric:* Dynamic quantifiable output evaluated during/after training.
• *Artifact:* Binary file generated during execution (weights, plots, logs).
• *Tag:* Key-value label for auditability (git hash, roll number, stage).

> **[SCREENSHOT PLACEHOLDER]**
> Figure 3.1: MLflow local UI comparing runs with distinct learning rates.
> *Expected file: reports/screenshots/3.1_mlflow_local.png*

3.2 DagsHub Remote Tracking Setup

Setup: Connected GitHub repository to DagsHub. Exported MLFLOW_TRACKING_URI and security token via .env. Verified that .env is excluded via .gitignore.

> **[SCREENSHOT PLACEHOLDER]**
> Figure 3.2: DagsHub project UI displaying remote tracking experiments.
> *Expected file: reports/screenshots/3.2_dagshub_setup.png*

3.3 Instrument VisionGuard Training (Manual vs Autolog)

Comparison: Manual logging (custom Keras callback) provides precise control over custom metrics and domain tags with negligible overhead. MLflow Autolog automatically captures model graphs and optimizer parameters, but generates verbose uncurated artifacts.

> **[SCREENSHOT PLACEHOLDER]**
> Figure 3.3: MLflow run overview showing parameters, metric curves, and metadata tags.
> *Expected file: reports/screenshots/3.3_run_page.png*

3.4 Artifact Logging

What was logged: The trained .keras model binary, confusion matrix plots, ROC curves, split text files, and pinned dependencies were uploaded directly to the MLflow artifact repository.

> **[SCREENSHOT PLACEHOLDER]**
> Figure 3.4: MLflow Artifacts tab displaying saved model and forensic assets.
> *Expected file: reports/screenshots/3.4_artifacts_tab.png*

3.5 Experiment Comparison (6 Runs)

Comparison Matrix: 6 systematic runs explored variations in backbone, learning rate, augmentation, and image size. Fine-tuned MobileNetV2 achieved peak ROC-AUC (0.9835) and was promoted to champion.

[SCREENSHOT PLACEHOLDER]
Figure 3.5: MLflow parallel coordinates / metric comparison chart across 6 runs.
Expected file: reports/screenshots/3.5_comparison_view.png

3.6 Model Registry & Champion Alias

Registry Governance: The best model was registered under VisionGuard_Champion. Two versions were committed, and version 2 was assigned the champion alias. src/registry_test.py dynamically resolved and executed single-image inference via this alias.

[SCREENSHOT PLACEHOLDER]
Figure 3.6: MLflow Model Registry UI displaying registered versions and champion alias.
Expected file: reports/screenshots/3.6_model_registry.png

Chapter 4: Tasks 4 & 5 - Flask Web Application & Database Trail

4.1 Inference Engine Module & Optimization

Architectural Design: The model is loaded once at server startup as a Singleton. Loading weights per request introduces unacceptable disk I/O latency (2-5s) and leaks memory under concurrency. Zero disk writes are enforced: incoming images are validated and processed in-memory byte buffers.

> **[SCREENSHOT PLACEHOLDER]**
> Figure 4.1: Terminal output demonstrating inference latency and verdict on test images.
> *Expected file: reports/screenshots/4.1_inference_console.png*

4.2 Route Security, Validation & Friendly Error Pages

Security Checks: Strict MIME and extension validation (JPG/PNG/WEBP), maximum payload restriction (5MB), filename sanitization via secure_filename, and byte-level PIL decode verification protect against denial-of-service and arbitrary file execution.

> **[SCREENSHOT PLACEHOLDER]**
> Figure 4.2: Friendly error responses for wrong extension, size overflow (413), and decode corruption.
> *Expected file: reports/screenshots/4.2_errors_combined.png*

4.3 Frontend Interface & 4.4 Production Configuration

Operational Rigor: Built responsive HTML/CSS dashboard displaying forensic verdict, confidence, and caveats. Flask debug mode is explicitly disabled (debug=False) to prevent arbitrary code execution vulnerabilities.

> **[SCREENSHOT PLACEHOLDER]**
> Figure 4.3: VisionGuard web dashboard displaying forensic classification.
> *Expected file: reports/screenshots/4.3_frontend.png*

> **[SCREENSHOT PLACEHOLDER]**
> Figure 4.4: JSON health check response from /health.
> *Expected file: reports/screenshots/4.4_health_check.png*

Task 5: Database Audit Trail

Audit Compliance: Every query writes an immutable record to SQLite (timestamp, filename, verdict, confidence, model identifier, latency). Accessible through the /history interface.

> **[SCREENSHOT PLACEHOLDER]**
> Figure 5.1: Database audit trail table displaying recorded historical queries.
> *Expected file: reports/screenshots/5.1_history_table.png*

Chapter 5: Task 6 - Containerization with Docker & Docker Compose

6.1 Docker Warm-Up & Core Concept Definitions

Concepts:

• *Image:* Immutable, read-only template with runtime instructions and dependencies.
• *Container:* Executable isolated runtime instance of an image.
• *Layer:* Read-only filesystem delta produced by a Dockerfile instruction.
• *Volume:* Persistent host-managed storage decoupled from container lifecycle.
• *Port Mapping:* Forwarding host network interface ports to isolated container ports.

> **[SCREENSHOT PLACEHOLDER]**
> Figure 6.1: Docker warmup CLI execution: run, logs, exec shell, and container prune.
> *Expected file: reports/screenshots/6.1_docker_warmup.png*

6.2 Optimized Dockerfile

Optimizations: Employs python:3.10-slim, installs requirements.txt prior to copying source code to maximize Docker cache utilization, executes as non-root user (appuser: UID 1000), incorporates native HEALTHCHECK, and utilizes tensorflow-cpu to cut image size by ~500MB.

> **[SCREENSHOT PLACEHOLDER]**
> Figure 6.2: Container shell showing execution under non-root appuser.
> *Expected file: reports/screenshots/6.2_dockerfile_user.png*

6.3 Multi-Service Compose Stack

Topology: Orchestrates web app and PostgreSQL database on an internal bridge network (visionguard_net). The database publishes no external host ports and validates readiness via pg_isready healthcheck before the web app boots. Model weights are mounted read-only (:ro).

> **[SCREENSHOT PLACEHOLDER]**
> Figure 6.3: Docker Compose status displaying healthy services and closed database port.
> *Expected file: reports/screenshots/6.3_compose_healthy.png*

6.4 Container Image Optimization Analysis

Size Reduction: Base unoptimized image measured 1.48 GB. Switching to Debian slim, purging pip cache (--no-cache-dir), excluding datasets via .dockerignore, and selecting CPU-only TensorFlow reduced final image size to 512 MB (a 65.4% reduction).

6.5 Operations and Troubleshooting Incident

Troubleshooting Simulation: The database password was intentionally mismatched in .env. The web container failed health check. Inspection of docker logs visionguard_web pinpointed FATAL: password authentication failed. The configuration was corrected and the stack recovered seamlessly.

[SCREENSHOT PLACEHOLDER]
Figure 6.5: Diagnostic docker logs showing error identification and resolution.
Expected file: reports/screenshots/6.5_troubleshooting_logs.png

6.6 Continuous Integration (CI) with GitHub Actions

CI Pipeline: Automated workflow runs Flake8 linting, multi-version Python testing (3.10, 3.11), and container builds on every push. Secrets are stored in encrypted repository secrets and never hardcoded in workflow YAMLs.

[SCREENSHOT PLACEHOLDER]
Figure 6.6: GitHub Actions CI status showing successful linting, testing, and container build.
Expected file: reports/screenshots/6.6_ci_runs.png

Chapter 6: Tasks 7 & 8 - API Cloud Deployment & Integration

Task 7: Cloud Model API Deployment

Deployment Architecture: The model inference engine was packaged with FastAPI and deployed to cloud container hosting with a publicly reachable HTTPS endpoint. Postman was used to test the public API from an external cellular network.

Public API Endpoint: https://shaheer-visionguard-api.hf.space/predict

> **[SCREENSHOT PLACEHOLDER]**
> Figure 7.1: Postman testing of public cloud API endpoint from external network.
> *Expected file: reports/screenshots/7.1_postman_cloud_api.png*

Task 8: Flask Integration with Cloud API

Integration Flow: Localhost Flask forwards uploaded image bytes directly to the Cloud API over HTTPS, parses JSON forensic responses, displays verdicts on the UI, and logs records to the local database audit trail.

> **[SCREENSHOT PLACEHOLDER]**
> Figure 8.1: Local Flask application successfully serving predictions via Cloud API.
> *Expected file: reports/screenshots/8.1_flask_cloud_integration.png*